



Implementing SPKF on a nonlinear model

- Refactor SPKF to represent real-time implementation better.
 - Initialization routine (initSPKF.m), called once at startup.
 - Update routine (iterSPKF.m), called every sample interval.
 - “Wrapper” code, coordinates the entire simulation process.
- Wrapper code begins (load data, store in local variables):

```
load simOut.mat model dT t u x z % Load model parameters and simulation profile

[nx,nt] = size(x); % Reserve storage for computed
xhatstore = zeros(nx,nt); % state estimates and bounds,
boundstore = zeros(nx,nt); % for plotting

SigmaX0 = 0.1*eye(nx,nx); % Define uncertainty of initial state
xhat0 = zeros(nx,1); % Define expected value of initial state

% Create spkfData structure and initialize variables
spkfData = initSPKF(xhat0,SigmaX0,u(1),dT,model);
```



Wrapper code (2): Main loop

- Wrapper code continues by entering main simulation loop.
 - “Measure” sensor readings; update SPKF.

```
% Now, enter loop for remainder of time, where we update the SPKF
% once per sample interval
hwait = waitbar(0,'Computing...');
for k = 2:length(t)
    uk = u(k); % "measure" input force
    zk = z(k); % "measure" nonlinear output

    % Compute estimate of present state vector and its bounds
    [xhatstore(:,k),boundstore(:,k),spkfData] = iterSPKF(uk,zk,dT,spkfData);

    % update waitbar periodically, but not too often (slow procedure)
    if mod(k,100) == 0
        waitbar(k/length(t),hwait);
    end
end
close(hwait);
```



Wrapper code (3): Plot state and estimate

- Wrapper code continues with plotting code.
- First, we plot the states, estimates, and confidence bounds.

```
% Plot state and estimate, with bounds
figure(1); clf; CL = lines;
t2 = [t fliplr(t)]; % Prepare for plotting bounds via "fill"
x2 = [xhatstore-boundstore fliplr(xhatstore+boundstore)];
h1 = fill(t2,x2,CL(1,:), 'FaceAlpha',0.5, 'LineStyle','none'); hold on; grid on
set(gca, 'ColorOrderIndex',1);
h2 = plot(t,x',t,xhatstore,'--'); ylim([-0.2 0.25]);
legend([h2;h1(1)],{'True position','True velocity','Position estimate',...
    'Velocity estimate','Bounds'},'NumColumns',3);
title('Demonstration of SPKF filter state estimates');
xlabel('Time (s)'); ylabel('State (m or m/s)');
```



Wrapper code (4): Plot errors

- Next, we plot estimation error and output some statistics.

```
% Plot estimation error with confidence bounds.
figure(2); clf
for k = 1:nx
    subplot(nx,1,k);
    fill(t2,[-boundstore(k,:) flipplr(boundstore(k,:))], 'b',...
        'FaceAlpha',0.1,'LineStyle','none'); hold on; grid on;
    plot(t,xerr(k,:), 'b');
    title(sprintf('State %d estimation error with bounds',k));
    hx = xlabel('Time (s)'); ylabel('Error');
end

% Compute and output some statistics
spkfErr = x - xhatstore;
spkfBnd = boundstore;
fprintf('RMS state estimation error = %g\n',rms(spkfErr(:)));
ind = find(abs(spkfErr)>spkfBnd);
fprintf('Time error outside bounds = %g%\n',length(ind)/numel(x)*100);
```



SPKF initialization code

- The SPKF initialization code sets up a data structure named `spkfData` that is updated recursively in the main loop.

```
function spkfData = initSPKF(xhat0,SigmaX0,priorU,dT,model)
    spkfData.xhat = xhat0; % Initial state description
    spkfData.SigmaX = SigmaX0; % Initial estimation-error covariance
    spkfData.SigmaV = model.SigmaV; % Sensor-noise covariance
    spkfData.SigmaW = model.SigmaW; % Process-noise covariance
    spkfData.Sw = model.Sw; % Cts-time process-noise covariance
    spkfData.priorU = priorU; % Previous value of input force
    spkfData.model = model; % Store model data structure too
    spkfData.Qbump = 5; % In case SigmaX collapses

    spkfData.Snoise = real(chol(diag([spkfData.Sw/dT; spkfData.SigmaV]),'lower'));
    % Alternate implementation with matrix SigmaW
    % spkfData.Snoise = real(chol(blkdiag(spkfData.SigmaW,spkfData.SigmaV),...
    % 'lower'));

    % previous value of input
    spkfData.priorU = priorU;
```



SPKF initialization code (cont.)

- The SPKF initialization code continues by defining the weights used in the updates.

```
% SPKF specific parameters
nx = length(spkfData.xhat); spkfData.nx = nx; % state-vector length
nz = 1; spkfData.nz = nz; % measurement-vector length
nu = 1; spkfData.nu = nu; % input-vector length
nw = size(model.Sw,1); spkfData.nw = nw; % process-noise-vector length
% Alternate implementation with matrix SigmaW
% nw = size(model.SigmaW,1); spkfData.nw = nw; % process-noise-vector length
nv = size(model.SigmaV,1); spkfData.nv = nv; % sensor-noise-vector length
na = nx+nw+nv; spkfData.na = na; % augmented-state-vector length

h = sqrt(3); spkfData.h = h; % SPKF/CDKF tuning factor
Weight1 = (h*h-na)/(h*h); % weighting factors when computing mean
Weight2 = 1/(2*h*h); % and covariance
spkfData.Wm = [Weight1; Weight2*ones(2*na,1)]; % mean
spkfData.Wc = spkfData.Wm; % covar

spkfData.model = model; % store model data structure too
end
```



Summary

- Implementation of SPKF on nonlinear model refactors code.
 - Initialization routine (`initSPKF.m`), called once at startup.
 - Update routine (`iterSPKF.m`), called every sample interval.
 - “Wrapper” code, coordinates the entire simulation process.
- You have now seen the details of the initialization routine and the wrapper code.
- Next lesson will present the update routine, `iterSPKF.m`.